Lab 4 AVCV 2026/2027: Fire-risk classification from satellite images

CUSTOM and FINE-TUNING networks on the FireRisk subset

Daniel Truzman (100475161) César Álvarez-Cascos (100475191) • CodaBench team `100475161_100475191`

1. INTRODUCTION

The goal of this lab is to assign one of seven FireRisk classes (Water, Very Low, Low, Moderate, High, Very High and Non-burnable) to a single RGB satellite image. It is an image-classification problem, not fire detection: the label describes the fire risk of the area, not whether a fire is visible. The task has two main difficulties. The classes are strongly imbalanced, from 2.4% of the images for Water to 31% for Very Low, so a model that ignores the rare classes can still look accurate; this is why the lab uses macro-F1, which weights every class equally. Moreover, the five risk levels are ordered and look alike, and neighbouring levels such as Low, Moderate and High turn out to be the main source of errors.

We work with a subset of FireRisk [1] that contains 5,861 images of 320×320 pixels. It is split in a stratified way into 3,516 training, 1,172 validation and 1,173 test images with hidden labels (class counts in Table 3). The networks are trained on 224×224 crops of these images (Section 2.1).

The CUSTOM system is a VGG-style CNN trained from scratch and refined through a staged random search; the FINE-TUNING (FT) system adapts ImageNet-pretrained torchvision models, of which Swin-T was selected. The experiments tried to answer a few concrete questions: whether the network from scratch was limited by capacity or by the data, how class weighting and augmentation affect the minority classes, and whether ensembles or retraining on train+val would help.

2. METHODOLOGY

2.1. Preprocessing and data augmentation

Images are read as RGB tensors in $[0, 1]$ and normalised with the ImageNet statistics, which the pretrained backbones require; CUSTOM uses the same pipeline. During training we take a random 224×224 crop of each 320×320 image and augment every batch on the GPU with horizontal and vertical flips, 90° rotations and small random changes of gamma, brightness, contrast and saturation, with a *light* and a *strong* preset (Table 1). An aerial image has no natural "up", so flips and rotations give new images of the same class for free, while the colour changes are kept mild so as not to alter the vegetation, probably the main cue for fire risk. Mixup [2], CutMix [3] and test-time augmentation (TTA) did not improve validation beyond the seed noise.

Validation images are centre-cropped to 224×224, the setting we used to select models. For the test predictions, however, we feed the whole 320×320 image, which both networks accept thanks to their final global pooling: even though they were trained and validated at 224×224, in our experiments the full image gave better results on the test set.

2.2. System design: CUSTOM network

The CUSTOM network (Table 5) uses only primitive layers: four VGG-style blocks [4] of 48 to 384 channels, each with two Conv 3×3–BatchNorm [5]–ReLU layers and a 2×2 max-pooling, followed by global average pooling (GAP) [6] and a small classifier with dropout (2.74 M parameters).

We built it step by step from the reference notebook. Its Flatten head held about 99% of the parameters, and in the first search phase every configuration with Flatten scored below all those with GAP, so GAP was our first change. Each 5×5 convolution became two stacked 3×3 ones with BatchNorm, which see the same area with fewer weights and train more stably. Some choices were taken from the literature instead of being searched, to save budget: BatchNorm, Kaiming initialisation, 3×3 kernels, AdamW and the cosine schedule with warm-up. Finally, since the network was still underfitting, we made it wider (Section 2.4).

2.3. System design: FINE-TUNING network

We kept the provided AlexNet [7] as a baseline and compared it with three more recent families of similar size (24–28 M parameters, so that they fit on our GPU), all with torchvision ImageNet-1k weights [8]: ConvNeXt-T [9], Swin-T [10] and EfficientNetV2-S [11]. Before the search, each network was trained for one epoch twice to check that the weights came out identical; EfficientNetV2-S failed this test (its loss became non-finite with mixed precision) and was replaced by ResNet-50 [12]. In every model the classifier is replaced by a new 7-output linear layer, and the adaptive pooling by a plain mean, which is deterministic on GPU.

We did not freeze any layer: satellite images are quite different from ImageNet photographs, so the whole network has to adapt. To avoid destroying the pretrained features, the backbone uses a small learning rate, while the

new head, which starts from random weights, uses the same rate or ten times more [13]. The inputs follow the pretraining conditions (224×224, ImageNet normalisation).

2.4. Training protocol and model selection

Table 1: Training configuration of the selected models.

	CUSTOM (n07_60ep)	FT (Swin-T, swin_p2_1)
Loss	Cross-entropy, class weights $\propto 1/\sqrt{n_c}$, no label smoothing, no mixup	
Optimiser & schedule	AdamW [14]; linear warm-up + cosine decay [15]; fp16 mixed precision	
Learning rate	$3.17 \cdot 10^{-4}$	backbone $2.37 \cdot 10^{-5}$, head $\times 1$
Weight decay	0.0464	0.0395
Batch / warm-up	128 / 1 epoch	32 / 2 epochs
Epochs (selected epoch)	60 (36)	30 (26)
Augmentation	strong	light, drop path 0.2
Common	Seed 42; full train split (no subsampling); Kaggle Tesla T4; PyTorch 2.11 [16]	

Table 1 gives the final configuration of both models. We reached it with a search in phases that went from broad to specific.

Why random search. We had around ten hyperparameters (architecture, learning rate, weight decay, batch size, augmentation, class weights, ...), so even two or three values each would make a grid far too large, and most of its runs would only vary hyperparameters that barely matter. Random search spreads the same budget over many more values of each one [17]: with 24 trials there is already a 71% chance that at least one falls in the best 5% of the search space. Bayesian optimisation would have been more efficient in the long run, but each trial depends on the previous results, which did not fit our setup of two computers running independently. To save time, a trial was stopped at epochs 4 and 8 if it was below the median of the previous ones [18].

CUSTOM. Phase 1 (24 trials of 15 epochs, shared between the two computers) chose what matters most: architecture, optimiser and learning rate. Phase 2 (16 trials) tuned the regularisation and secondary settings around the winner. These short runs only served to compare configurations, so in phase 3 the three best were retrained from scratch for 40 epochs and compared by their mean validation F1 over the last 10 epochs, which depends less on one lucky epoch. All three got stuck at about 0.45–0.46 with almost equal training and validation scores: the network was underfitting, and more tuning would not fix that. In phases 4 and 5 we therefore changed one thing at a time on the best configurations. Only two changes clearly helped, a wider network (48 base channels) and 60 instead of 40 epochs, whereas inverse-frequency class weights made things worse and more channels or blocks did not help. A different seed alone moves the score by about 0.008, so smaller differences were treated as noise. In parallel, a broader 121-trial search found a model with a higher validation score, but it did not hold on the public test, so we kept the simpler network.

FINE-TUNING. Pretrained networks converge much faster, so a lighter search was enough: 10 configurations of 10 epochs per backbone, after which the best two of each were retrained for 30 epochs. Swin-T obtained the best validation macro-F1 (0.546) and overfitted much less than ResNet-50 (training F1 0.77 against 0.99), so it is our FT model.

Selection and reproducibility. In every run we keep the epoch with the best validation macro-F1, without early stopping. Reproducibility was a priority from the start: all seeds are fixed, including those of the data-loading workers, and PyTorch runs in deterministic mode, so under the declared conditions (Table 1) the notebook reproduces the selected models bit by bit. Small differences can only appear on another GPU model (different kernels and order of the sums), on another operating system (Windows and Linux use a different number of loading workers, which changes the random crops) or on CPU.

3. RESULTS AND ERROR ANALYSIS

Our main metric is macro-F1, the mean of the seven per-class F1 scores. All model choices were made on validation macro-F1, with training F1 only as a diagnostic; the public leaderboard was only used to discard the deep-search model and to choose the 320×320 test input.

Experiments. Table 2 summarises the main experiments. For CUSTOM, the jump from the reference ($\approx$0.36) to the final network (0.502) came from the GAP head, the extra width and the longer training. Other ideas we tried at the end, such as CutMix, ensembles, per-class decision thresholds or retraining on train+val, gave no clear gain. The thresholds were a useful lesson: they raised the score on the validation set, but almost nothing under cross-validation, so they were simply overfitting it. For FT, all modern backbones beat AlexNet (0.452).

Learning curves. The CUSTOM curves in Figure 3 stay close together and validation F1 flattens at about 0.50 after epoch 30: the network does not overfit, it just stops learning, which again points to a limit of the data rather than of the model. Swin-T fits the training set much better (0.77) but gains only 0.04 on validation.

Per-class results. Water and Non-burnable are easy for both models (F1 of 0.80 or more), while Low, Moderate

and High stay between 0.2 and 0.4 (Table 3). The confusion matrices (Figure 1) show that most mistakes happen among the risk levels, and about half of them between two neighbouring levels: the models tend to get close to the right level but often miss it by one. Low is the clearest case, with 40% of its images labelled Very Low by CUSTOM and 35% by Swin-T.

The examples in Figure 2 suggest that part of these errors cannot be solved from the image alone. The Low example is mostly water and both models say Water, while the Very High one shows pale, almost bare ground that both read as Very Low. The labels come from a hazard map much coarser than the image [1], so what makes an area risky is often not visible in it. Consistently, CUSTOM reaches only 0.53 even on its own training images.

Competition feedback. Our final submission (ID 1042782, Table 4) scores 0.50 for CUSTOM and 0.49 for FT. The CUSTOM network keeps its validation score, whereas the models that came out of the longest searches lose the most: Swin-T drops from 0.546 to 0.49, and the best deep-search model from 0.524 to 0.47. Reusing the same 1,172 validation images for hundreds of decisions makes their scores optimistic, and the public subset is small and noisy.

4. CONCLUSIONS

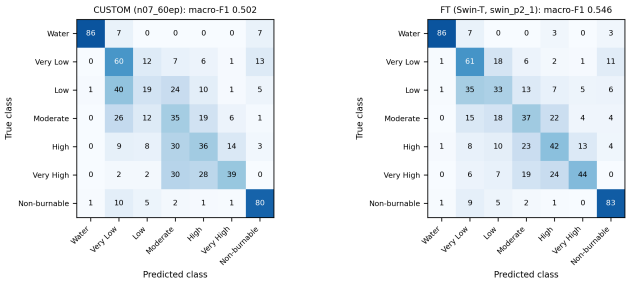
For CUSTOM, the key step was realising that the network was underfitting: once we stopped tuning the regularisation and gave it more width and more epochs, validation macro-F1 went from 0.44 to 0.50. For FT, the pretraining did most of the work, and with a small backbone learning rate Swin-T reached 0.546 in 30 epochs. Fine-tuning is clearly better on validation, but on the public test both systems end almost level (0.50 vs 0.49), so with this little data a well-designed small network trained from scratch is still a competitive and much cheaper option.

The main limit is the data: the intermediate classes are hard even on training images, in line with the 61–65% accuracy reported with about 91k images in the original paper [1]. With more time we would use k-fold cross-validation for a more honest selection, a loss that takes into account the order of the risk levels, and pretraining on satellite images.

Individual contributions.

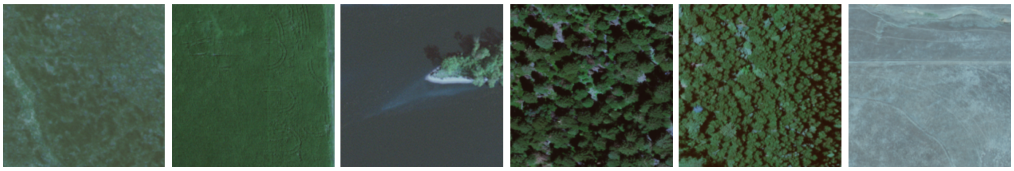
- **César Álvarez-Cascos (100475191):** design of CUSTOM search phases 1–3 (two-machine random search, finalists and the 121-trial deep search) and the GPU speed-up of training (images preloaded in RAM, batch augmentation on the GPU, mixed precision).
- **Daniel Truzman (100475161):** design of CUSTOM phases 4–5 (widening to 48×2, 60 epochs, selection of n07_60ep, deterministic Kaggle runs) and the CutMix, ensemble, threshold and train+val experiments.
- **Both:** the training runs on both members' machines, the notebook and the fine-tuning search.

Use of AI tools. We used an AI assistant as a productivity tool, always following our own criteria and instructions once we had defined the design criteria ourselves. It helped us implement and debug code, look for errors, suggest possible improvements and edit the text of this report. Every experiment, result and decision was run, checked and taken by the authors.



(a) CUSTOM **(b)** FT (Swin-T)

Figure 1: Validation confusion matrices (224 × 224 centre crop), row-normalised (%); rows: true class, columns: predicted class.



True	Water	Very Low	Low	Moderate	High	Very High
CUSTOM	Very Low	Non-burnable	Water	Very High	Very High	Very Low
FT	High	Non-burnable	Water	Very High	Moderate	Very Low

Figure 2: Validation errors shared by both models: per true class, the one with the most confident FT prediction (chosen automatically).

Table 2: Main experiments: validation macro-F1 at the best epoch (L10: mean of the last 10 epochs). Reference and selected models in bold.

Cat.	Change	Objective	Budget	Val F1
CUSTOM	**Reference notebook (StepLR)**	Baseline	60 ep	≈0.36
CUSTOM	GAP instead of Flatten (first version)	Fewer FC parameters; train F1 ≈0.42	30 ep	0.440
CUSTOM	Phases 1–3 random search	Architecture, optimiser, regularisation	15 ep (finalists 40)	0.45–0.465
CUSTOM	Inverse-frequency class weights	Help minority classes	40 ep	L10 0.431 vs 0.453
CUSTOM	48 ch, 2 convs/block (p4_09_B)	More capacity (underfitting)	40 ep	0.496 (L10 0.483)
CUSTOM	64 ch; 5 blocks; crop 288/eval 320	More capacity / resolution	40 ep	no gain
CUSTOM	**60 epochs (n07_60ep)**	Longer schedule	60 ep	**0.502** (L10 0.489)
CUSTOM	CutMix (standard; risk-only)	Regularisation	80 ep	L10 0.483; 0.470 vs 0.486
CUSTOM	Deep search (F3_B_038)	Joint search, 121 trials	80 ep, input 320	0.524
FT	**AlexNet (provided recipe)**	Baseline	5 ep	0.452
FT	ConvNeXt-T (cnx_p2_0)	Modern CNN backbone	30 ep	0.505
FT	ResNet-50 (r50_p2_0)	Residual backbone (train F1 0.99)	30 ep	0.539
FT	**Swin-T (swin_p2_1)**	Transformer backbone	30 ep	**0.546**

Table 3: Class counts and per-class validation precision / recall / F1 of the selected models (224 × 224 centre crop, as used for model selection).

Class	Train	Val	CUSTOM P/R/F1	FT P/R/F1
0 Water	86	29	.83/.86/.85	.81/.86/.83
1 Very Low	1,088	363	.59/.60/.60	.65/.61/.63
2 Low	535	178	.27/.19/.22	.33/.33/.33
3 Moderate	431	144	.29/.35/.32	.39/.37/.38
4 High	315	105	.31/.36/.33	.39/.42/.41
5 Very High	163	54	.41/.39/.40	.41/.44/.43
6 Non-burnable	898	299	.79/.80/.80	.80/.83/.81
Total / macro	3,516	1,172	.500/.508/**.502**	.539/.553/**.546**
Accuracy			.535	.577

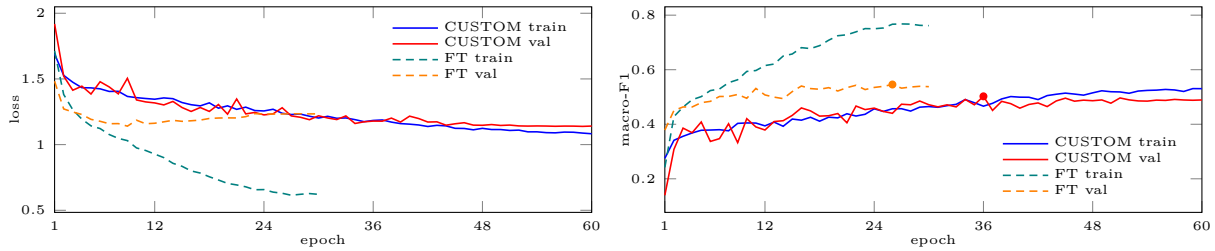
Table 4: Public CodaBench macro-F1 (fixed undisclosed test subset). Bold: final submission with the selected models; n/r: not recorded.

ID	Date	Models (custom / ft)	CUSTOM	FT
1036062	05 Nov	p4_09_B / placeholder	0.49	–
1038020	06 Nov	n07, train+val / –	0.49	–
1038186	06 Nov	F3_B_038 / F1_B_037	0.47	0.47
1038323	06 Nov	CUSTOM ens. (5) / + TTA	0.48	n/r
1042782	**08 Nov**	**n07_60ep / swin_p2_1**	**0.50**	**0.49**

Table 5: Final CUSTOM architecture.

Stage	Layers	Output
Input	RGB crop	3 × 224 × 224
Blocks 1–4	[Conv3×3-BN-ReLU]×2, MaxPool 2×2	48/96/192/384 ch
	(spatial size 112/56/28/14)	384 × 14 × 14
GAP	spatial mean	384
Head	FC 384→256, ReLU, Dropout 0.3, FC 256→7	7

2,739,447 parameters; Kaiming initialisation.



(a) Loss **(b)** Macro-F1

Figure 3: Train (with augmentation) and validation curves of the selected models; dots mark the selected epochs (CUSTOM 36, FT 26).

REFERENCES

[1] Shuchang Shen, Sachith Seneviratne, Xinye Wanyan, et al. FireRisk: A remote sensing dataset for fire risk assessment with benchmarks using supervised and self-supervised learning. In *DICTA*, 2023.

[2] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, et al. mixup: Beyond empirical risk minimization. In *ICLR*, 2018.

[3] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, et al. CutMix: Regularization strategy to train strong classifiers with localizable features. In *ICCV*, 2019.

[4] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[6] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *ICLR*, 2014.

[7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012.

[8] Olga Russakovsky, Jia Deng, Hao Su, et al. ImageNet large scale visual recognition challenge. *IJCV*, 2015.

[9] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, et al. A ConvNet for the 2020s. In *CVPR*, 2022.

[10] Ze Liu, Yutong Lin, Yue Cao, et al. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.

[11] Mingxing Tan and Quoc V. Le. EfficientNetV2: Smaller models and faster training. In *ICML*, 2021.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, et al. Deep residual learning for image recognition. In *CVPR*, 2016.

[13] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *ACL*, 2018.

[14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[15] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

[16] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[17] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *JMLR*, 2012.

[18] Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, et al. Google vizier: A service for black-box optimization. In *KDD*, 2017.